

Axiomatic System

Axiom 1 (Infinite Adjunction).

Beginning with a base set P_0 , whenever a set P_n has been constructed, there exists a fixed distinguished element

$$a_{n+1} \notin P_n$$

which has not appeared at any preceding construction stage. Distinct stages use distinct elements. Hence the construction may continue indefinitely.

At this point the subscripts

$$0, 1, 2, \dots$$

serve only as construction-stage labels. Their arithmetic properties have not yet been assumed or defined.

Axiom 2 (Function Well-Definedness).

A function $f: X \rightarrow Y$ maps every input $x \in X$ to exactly one output $f(x) \in Y$. We denote evaluation by

$$f(x) \rightarrow y.$$

Axiom 3 (Structural Induction).

Let $\mathcal{C}$ be a family generated from a base object X_0 by repeated application of a well-defined function f . If a property holds for X_0 , and whenever it holds for a constructed object X_n it also holds for $f(X_n)$, then the property holds for every object generated from X_0 by repeated applications of f .

Definitions and Rules

1.1 Base Set

Let

$$P_0 = \emptyset.$$

The empty set serves as the base representation of the symbol

$$0.$$

1.2 Canonical Finite Sets

Using Axiom 1, define the canonical sets recursively.

Whenever P_n has already been constructed, define

$$P_{n+1} = P_n \cup \{a_{n+1}\},$$

where

$$a_{n+1} \notin P_n.$$

Consequently,

$$P_0 = \emptyset,$$

$$P_1 = \{a_1\},$$

$$P_2 = \{a_1, a_2\},$$

$$P_3 = \{a_1, a_2, a_3\},$$

and so on.

Let $\mathcal{P}$ denote the smallest family containing P_0 and closed under this construction.

Thus

$$\mathcal{P} = \{P_0, P_1, P_2, P_3, \dots\}.$$

Because the element introduced at each stage is fixed, each construction stage has a unique output.

1.3 Output Symbols and the Cardinality Function

We now assign one output symbol to each construction stage.

Define Θ on the canonical family $\mathcal{P}$ recursively by

$$\Theta(P_0) = 0,$$

and whenever P_{n+1} is the immediate successor of P_n , let $\Theta(P_{n+1})$ be the next output symbol after $\Theta(P_n)$.

Accordingly,

$$\Theta(P_0) = 0,$$

$$\Theta(P_1) = 1,$$

$$\Theta(P_2) = 2,$$

$$\Theta(P_3) = 3,$$

and so on.

Hence, after these symbols have been generated,

$$\Theta(P_n) = n.$$

The statement $\Theta(P_n) = n$ therefore does not use a previously defined arithmetic operation. It associates the symbol generated at stage n with the canonical set generated at the same stage.

Since each P_n contains exactly the elements introduced during the first n nonzero construction stages, $\Theta(P_n)$ is also the finite cardinality of P_n . Therefore, once the symbols have been constructed, we may write

$$\Theta(P_n) = |P_n|.$$

The domain of Θ in this construction is $\mathcal{P}$, so Θ is never required to assign a finite natural-number output to an arbitrary infinite set.

1.4 Successor Function

Define

$$\Sigma : \mathcal{P} \rightarrow \mathcal{P}$$

by

$$\Sigma(P_n) = P_n \cup \{a_{n+1}\} = P_{n+1}.$$

Because a_{n+1} is fixed at construction stage $n + 1$, Σ maps every canonical set to a unique canonical set and is therefore well-defined.

Applying Σ repeatedly gives

$$P_0 = \emptyset,$$

$$\Sigma(P_0) = P_1 = \{a_1\},$$

$$\Sigma(\Sigma(P_0)) = P_2 = \{a_1, a_2\},$$

$$\Sigma(\Sigma(\Sigma(P_0))) = P_3 = \{a_1, a_2, a_3\}.$$

Evaluating with Θ ,

$$\Theta(P_0) = 0,$$

$$\Theta(\Sigma(P_0)) = 1,$$

$$\Theta(\Sigma(\Sigma(P_0))) = 2,$$

$$\Theta(\Sigma(\Sigma(\Sigma(P_0)))) = 3.$$

For any already constructed nonnegative output symbol k , write

$$\Sigma^k(P)$$

for k repeated applications of Σ , with

$$\Sigma^0(P) = P.$$

1.5 Natural Numbers Definition

The natural numbers are defined to be exactly the output symbols produced by evaluating Θ on the canonical sets:

$$\mathbb{N} = \{\Theta(P) \mid P \in \mathcal{P}\}.$$

Thus

$$\mathbb{N} = \{0, 1, 2, 3, \dots\}.$$

This definition does not require a statement of the form $k \in \mathbb{N}$ before $\mathbb{N}$ itself has been constructed.

Now that $\mathbb{N}$ has been constructed, the evaluation function may be written formally as

$$\Theta : \mathcal{P} \rightarrow \mathbb{N}.$$

Defining Addition

We define addition using the canonical sets, the successor function Σ , and the evaluation function Θ .

For $P_1, P_2 \in \mathcal{P}$, define

$$A(P_1, P_2) = \Theta(\Sigma^{\Theta(P_2)}(P_1)).$$

Thus $A(P_1, P_2)$ is a natural-number symbol, not a set.

We define the ordinary notation $+$ by

$$\Theta(P_1) + \Theta(P_2) := A(P_1, P_2).$$

Therefore,

$$\Theta(P_1) + \Theta(P_2) = \Theta(\Sigma^{\Theta(P_2)}(P_1)).$$

Addition by Zero

Since

$$\Theta(P_0) = 0,$$

we have

$$A(P_1, P_0) = \Theta(\Sigma^{\Theta(P_0)}(P_1)) = \Theta(\Sigma^0(P_1)) = \Theta(P_1).$$

Hence

$$\Theta(P_1) + 0 = \Theta(P_1).$$

Also,

$$A(P_0, P_1) = \Theta \left(\Sigma^{\Theta(P_1)}(P_0) \right).$$

By construction, applying Σ to P_0 exactly $\Theta(P_1)$ times produces P_1 . Therefore

$$A(P_0, P_1) = \Theta(P_1),$$

so

$$\boxed{0 + \Theta(P_1) = \Theta(P_1)}.$$

Successor Compatibility

Suppose

$$A(P_1, P_2) = \Theta(P_r).$$

Then

$$\Sigma^{\Theta(P_2)}(P_1) = P_r.$$

Since $\Sigma(P_2)$ contains exactly one more constructed element than P_2 ,

$$\Sigma^{\Theta(\Sigma(P_2))}(P_1) = \Sigma \left(\Sigma^{\Theta(P_2)}(P_1) \right).$$

Therefore

$$A(P_1, \Sigma(P_2)) = \Theta \left(\Sigma \left(\Sigma^{\Theta(P_2)}(P_1) \right) \right) = \Theta(\Sigma(P_r)).$$

Likewise,

$$\Sigma^{\Theta(P_1)}(\Sigma(P_2)) = \Sigma \left(\Sigma^{\Theta(P_1)}(P_2) \right).$$

Thus taking one successor step in either input produces one successor step in the resulting canonical set.

Commutativity of Addition

Theorem 1. *For all $P_1, P_2 \in \mathcal{P}$,*

$$A(P_1, P_2) = A(P_2, P_1).$$

Consequently,

$$\Theta(P_1) + \Theta(P_2) = \Theta(P_2) + \Theta(P_1).$$

Proof. Fix P_1 . We use structural induction on P_2 .

For the base case,

$$P_2 = P_0.$$

Then

$$A(P_1, P_0) = \Theta(P_1).$$

Also,

$$A(P_0, P_1) = \Theta(\Sigma^{\Theta(P_1)}(P_0)) = \Theta(P_1).$$

Hence

$$A(P_1, P_0) = A(P_0, P_1).$$

Now assume

$$A(P_1, P_2) = A(P_2, P_1).$$

Let

$$P'_2 = \Sigma(P_2).$$

By successor compatibility,

$$A(P_1, P'_2)$$

is the output symbol immediately following

$$A(P_1, P_2).$$

Likewise,

$$A(P'_2, P_1)$$

is the output symbol immediately following

$$A(P_2, P_1).$$

By the induction hypothesis,

$$A(P_1, P_2) = A(P_2, P_1),$$

so their immediate successor symbols are equal. Therefore

$$A(P_1, P'_2) = A(P'_2, P_1).$$

By structural induction,

$$A(P_1, P_2) = A(P_2, P_1)$$

for every $P_2 \in \mathcal{P}$.

□

Therefore it is valid to write

$$\boxed{\Theta(P_1) + \Theta(P_2) = \Theta(P_2) + \Theta(P_1).}$$

Example: $1 + 2$

Let

$$P_0 = \emptyset,$$

$$P_1 = \Sigma(P_0),$$

$$P_2 = \Sigma(\Sigma(P_0)).$$

Then

$$\Theta(P_1) = 1, \quad \Theta(P_2) = 2.$$

Evaluate

$$A(P_1, P_2) = \Theta \left(\Sigma^{\Theta(P_2)}(P_1) \right).$$

Since

$$\Theta(P_2) = 2,$$

we obtain

$$A(P_1, P_2) = \Theta(\Sigma^2(P_1)).$$

But

$$\Sigma^2(P_1) = \Sigma(\Sigma(P_1)) = P_3.$$

Thus

$$A(P_1, P_2) = \Theta(P_3) = 3.$$

Therefore

$$1 + 2 = 3.$$

By the commutativity theorem,

$$A(P_2, P_1) = A(P_1, P_2) = 3,$$

so

$$2 + 1 = 3.$$

Hence

$$\boxed{1 + 2 = 2 + 1 = 3.}$$

Subtraction on Natural Numbers via Element Removal

We now define subtraction when the result remains nonnegative.

1. Element Removal Function

Define the removal function N on every nonempty canonical set by the predecessor rule

$$\boxed{N(P_{n+1}) = P_n.}$$

Here $n+1$ denotes the immediate next construction-stage label already used in the canonical-set construction. No subtraction operation is being assumed in this definition. Equivalently, N removes the distinguished element introduced most recently in the canonical construction.

For example,

$$P_3 = \{a_1, a_2, a_3\}.$$

Then

$$N(P_3) = \{a_1, a_2\} = P_2.$$

Similarly,

$$N(P_2) = P_1$$

and

$$N(P_1) = P_0.$$

In particular,

$$N(P_1) = P_0 = \emptyset,$$

and therefore

$$\Theta(N(P_1)) = 0.$$

The rule specifies a unique element to remove, so N is well-defined.

2. Iterated Removal

For any natural-number symbol k , define the iteration recursively by

$$\boxed{N^0(P) = P.}$$

Whenever $N^k(P)$ has already been constructed and is nonempty, define the next iteration by

$$\boxed{N^{k+1}(P) = N(N^k(P)) .}$$

Thus iterated removal is defined using the already-constructed successor progression of natural-number symbols and does not assume subtraction.

For example,

$$N^2(P_3) = N(N(P_3)) = N(P_2) = P_1.$$

3. Ordering of Natural Numbers

Before restricting subtraction, define the order relation $\leq$.

For canonical sets P_1, P_2 , define

$$\boxed{\Theta(P_2) \leq \Theta(P_1)}$$

to mean that P_1 can be obtained from P_2 by zero or more applications of Σ .

Equivalently,

$$\Theta(P_2) \leq \Theta(P_1)$$

when there exists a natural-number symbol k such that

$$\Sigma^k(P_2) = P_1.$$

Thus, for example,

$$1 \leq 3$$

because

$$\Sigma^2(P_1) = P_3.$$

4. Definition of Natural Subtraction

Let

$$\Delta(P_1, P_2)$$

be defined whenever

$$\Theta(P_2) \leq \Theta(P_1).$$

Define

$$\Delta(P_1, P_2) = \Theta \left(N^{\Theta(P_2)}(P_1) \right) .$$

We then define

$$\Theta(P_1) - \Theta(P_2) := \Delta(P_1, P_2)$$

whenever

$$\Theta(P_2) \leq \Theta(P_1).$$

The restriction guarantees that N is never required to remove an element from P_0 .

5. Example Evaluation

Let

$$P_3 = \{a_1, a_2, a_3\}$$

and

$$P_1 = \{a_1\}.$$

Then

$$\Theta(P_3) = 3$$

and

$$\Theta(P_1) = 1.$$

Evaluate

$$\Delta(P_3, P_1) = \Theta \left(N^{\Theta(P_1)}(P_3) \right) .$$

Since

$$\Theta(P_1) = 1,$$

we have

$$\Delta(P_3, P_1) = \Theta(N(P_3)).$$

By the definition of N ,

$$N(P_3) = P_2.$$

Therefore

$$\Delta(P_3, P_1) = \Theta(P_2) = 2.$$

Thus

$$\boxed{3 - 1 = 2.}$$

Constructing Negative Numbers

Natural-number subtraction as defined above does not yet assign an output to expressions such as

$$1 - 2.$$

We now construct signed integers using ordered pairs of canonical sets.

1. Ordered-Pair Representation

Take an ordered pair

$$(P_x, P_y),$$

where

$$P_x, P_y \in \mathcal{P}.$$

The first component records a positive contribution, and the second component records a negative contribution.

At this stage, however, the symbol “negative” describes the role of the second component only. A negative-number symbol such as -1 will be introduced only after the corresponding ordered-pair object has been constructed.

2. Reduction Function

Define

$$\Lambda[(P_x, P_y)]$$

recursively by

$$\Lambda[(P_x, P_y)] = \begin{cases} \Lambda[(N(P_x), N(P_y))], & \Theta(P_x) > 0 \text{ and } \Theta(P_y) > 0, \\ (P_x, P_y), & \Theta(P_x) = 0 \text{ or } \Theta(P_y) = 0. \end{cases}$$

Here

$$\Theta(P) > 0$$

means simply

$$P \neq P_0.$$

Every recursive step removes one element from both components.

Since P_x and P_y are finite canonical sets, after finitely many applications of N , at least one component must become P_0 .

Therefore Λ always terminates.

Most importantly,

$$\Lambda$$

always returns an ordered pair. Its output type does not change during the definition.

3. Reduced Signed Pairs

A pair

$$(P_x, P_y)$$

is called *reduced* if at least one component equals P_0 .

Thus every reduced pair has exactly one of the following forms:

$$(P_n, P_0),$$

$$(P_0, P_n),$$

or

$$(P_0, P_0).$$

The function Λ maps every ordered pair of canonical sets to exactly one reduced pair.

4. Signed Integer Notation

We now introduce notation for these already-constructed reduced pairs.

For a nonzero natural-number symbol n , define the notation

$$(P_n, P_0) \equiv n.$$

Define

$$(P_0, P_0) \equiv 0.$$

Finally, for a nonzero natural-number symbol n , introduce the new notation

$$\boxed{(P_0, P_n) \equiv -n.}$$

Thus the symbol

$$-n$$

is not used to construct the negative integer.

Instead, the ordered pair

$$(P_0, P_n)$$

is constructed first, and $-n$ is then assigned as notation for that object.

The signed integers are therefore the reduced pairs

$$\boxed{\mathbb{Z} = \{\Lambda[(P_x, P_y)] \mid P_x, P_y \in \mathcal{P}\}.$$

Using the notation above, these objects are written

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}.$$

5. Canonical Representation

Because Λ removes one element from each side simultaneously, every pair has a unique reduced representative.

For example,

$$\Lambda[(P_2, P_1)] = \Lambda[(N(P_2), N(P_1))] = \Lambda[(P_1, P_0)] = (P_1, P_0).$$

Thus

$$(P_2, P_1)$$

and

$$(P_1, P_0)$$

reduce to the same signed integer.

More generally, two ordered pairs

$$(P_a, P_b)$$

and

$$(P_c, P_d)$$

represent the same signed integer precisely when

$$\boxed{\Lambda[(P_a, P_b)] = \Lambda[(P_c, P_d)]}.$$

Hence signed integers have unique reduced representatives even though unreduced intermediate pairs may differ.

6. Examples

Example 1: $1 - 2$.

Take

$$P_1 = \{a_1\}$$

and

$$P_2 = \{a_1, a_2\}.$$

Evaluate

$$\Lambda[(P_1, P_2)].$$

Since both components are nonempty,

$$\Lambda[(P_1, P_2)] = \Lambda[(N(P_1), N(P_2))].$$

Using

$$N(P_1) = P_0$$

and

$$N(P_2) = P_1,$$

we obtain

$$\Lambda[(P_1, P_2)] = \Lambda[(P_0, P_1)].$$

Because the first component is now P_0 ,

$$\Lambda[(P_0, P_1)] = (P_0, P_1).$$

By the signed notation introduced above,

$$(P_0, P_1) \equiv -1.$$

Therefore

$$\boxed{1 - 2 = -1.}$$

Example 2: $2 - 1$.

Evaluate

$$\Lambda[(P_2, P_1)].$$

Since both components are nonempty,

$$\Lambda[(P_2, P_1)] = \Lambda[(N(P_2), N(P_1))].$$

Thus

$$\Lambda[(P_2, P_1)] = \Lambda[(P_1, P_0)] = (P_1, P_0).$$

By signed notation,

$$(P_1, P_0) \equiv 1.$$

Therefore

$$\boxed{2 - 1 = 1.}$$

Example 3: $2 - 2$.

Evaluate

$$\Lambda[(P_2, P_2)].$$

First,

$$\Lambda[(P_2, P_2)] = \Lambda[(N(P_2), N(P_2))] = \Lambda[(P_1, P_1)].$$

Both components remain nonempty, so

$$\Lambda[(P_1, P_1)] = \Lambda[(N(P_1), N(P_1))] = \Lambda[(P_0, P_0)].$$

The pair is now reduced, giving

$$\Lambda[(P_0, P_0)] = (P_0, P_0).$$

Since

$$(P_0, P_0) \equiv 0,$$

we obtain

$$\boxed{2 - 2 = 0.}$$

Addition and Subtraction of Signed Pairs

We now define arithmetic on signed-pair representations. The inputs may be reduced or unreduced; the final application of Λ produces the unique reduced representative.

Let

$$(P_a, P_b)$$

and

$$(P_c, P_d)$$

be signed-pair representations.

The positive contributions are represented by

$$P_a \quad \text{and} \quad P_c,$$

while the negative contributions are represented by

$$P_b \quad \text{and} \quad P_d.$$

Signed Addition

The combined positive count is

$$A(P_a, P_c).$$

The corresponding canonical set is

$$\Sigma^{A(P_a, P_c)}(P_0).$$

Likewise, the combined negative count is

$$A(P_b, P_d),$$

with corresponding canonical set

$$\Sigma^{A(P_b, P_d)}(P_0).$$

Therefore define

$$\boxed{\Gamma((P_a, P_b), (P_c, P_d)) = \Lambda \left[(\Sigma^{A(P_a, P_c)}(P_0), \Sigma^{A(P_b, P_d)}(P_0)) \right] .}$$

The result is a reduced signed pair. We therefore define ordinary addition notation for signed-pair representations by

$$\boxed{(P_a, P_b) + (P_c, P_d) := \Gamma((P_a, P_b), (P_c, P_d)) .}$$

Signed Additive Inverse

For a signed pair

$$(P_c, P_d),$$

define its additive inverse by exchanging its two components:

$$\boxed{-(P_c, P_d) := (P_d, P_c) .}$$

This is valid because the component that formerly contributed positively now contributes negatively, and the component that formerly contributed negatively now contributes positively.

For example,

$$-(P_2, P_0) = (P_0, P_2),$$

so the additive inverse of 2 is represented by

$$(P_0, P_2) \equiv -2.$$

Similarly,

$$-(P_0, P_2) = (P_2, P_0),$$

so the additive inverse of -2 is 2.

Signed Subtraction

Subtraction is addition of the additive inverse.

Thus

$$(P_a, P_b) - (P_c, P_d)$$

is defined as

$$(P_a, P_b) + (P_d, P_c).$$

Therefore

$$\Pi((P_a, P_b), (P_c, P_d)) = \Lambda \left[(\Sigma^{A(P_a, P_d)}(P_0), \Sigma^{A(P_b, P_c)}(P_0)) \right].$$

We therefore define ordinary subtraction notation for signed-pair representations by

$$(P_a, P_b) - (P_c, P_d) := \Pi((P_a, P_b), (P_c, P_d)).$$

1. Adding Same Signs: $(-2) + (-1)$

The inputs are

$$(P_0, P_2)$$

and

$$(P_0, P_1).$$

The new positive count is

$$A(P_0, P_0) = 0.$$

Therefore

$$P_{\text{pos}} = \Sigma^0(P_0) = P_0.$$

The new negative count is

$$A(P_2, P_1) = 3.$$

Therefore

$$P_{\text{neg}} = \Sigma^3(P_0) = P_3.$$

Hence

$$\Gamma((P_0, P_2), (P_0, P_1)) = \Lambda[(P_0, P_3)].$$

Since the first component is already P_0 ,

$$\Lambda[(P_0, P_3)] = (P_0, P_3).$$

By signed notation,

$$(P_0, P_3) \equiv -3.$$

Therefore

$$\boxed{(-2) + (-1) = -3.}$$

2. Adding Opposite Signs: $(-2) + 1$

The inputs are

$$(P_0, P_2)$$

and

$$(P_1, P_0).$$

The new positive count is

$$A(P_0, P_1) = 1,$$

so

$$P_{\text{pos}} = P_1.$$

The new negative count is

$$A(P_2, P_0) = 2,$$

so

$$P_{\text{neg}} = P_2.$$

Thus

$$\Gamma((P_0, P_2), (P_1, P_0)) = \Lambda[(P_1, P_2)].$$

Applying N once to both components gives

$$\Lambda[(P_1, P_2)] = \Lambda[(P_0, P_1)] = (P_0, P_1).$$

Since

$$(P_0, P_1) \equiv -1,$$

we obtain

$$\boxed{(-2) + 1 = -1.}$$

3. Subtracting Same Signs: $1 - 2$

The inputs are

$$(P_1, P_0)$$

and

$$(P_2, P_0).$$

Using the subtraction definition,

$$A(P_1, P_0) = 1,$$

so

$$P_{\text{pos}} = P_1.$$

Also,

$$A(P_0, P_2) = 2,$$

so

$$P_{\text{neg}} = P_2.$$

Thus

$$\Pi((P_1, P_0), (P_2, P_0)) = \Lambda[(P_1, P_2)].$$

Reduction gives

$$\Lambda[(P_1, P_2)] = \Lambda[(P_0, P_1)] = (P_0, P_1).$$

Since

$$(P_0, P_1) \equiv -1,$$

we obtain

$$\boxed{1 - 2 = -1.}$$

4. Subtracting Opposite Signs: $1 - (-2)$

The inputs are

$$(P_1, P_0)$$

and

$$(P_0, P_2).$$

The positive count is

$$A(P_1, P_2) = 3,$$

so

$$P_{\text{pos}} = P_3.$$

The negative count is

$$A(P_0, P_0) = 0,$$

so

$$P_{\text{neg}} = P_0.$$

Therefore

$$\Pi((P_1, P_0), (P_0, P_2)) = \Lambda[(P_3, P_0)].$$

Since the second component is already P_0 ,

$$\Lambda[(P_3, P_0)] = (P_3, P_0).$$

By signed notation,

$$(P_3, P_0) \equiv 3.$$

Hence

$$\boxed{1 - (-2) = 3.}$$

Proof by Fully Expanded Substitution

We now expand the preceding signed arithmetic computations back to the base set P_0 using only

$$\Sigma(P), \quad \Theta(P), \quad A(P_1, P_2), \quad N(P), \quad \Lambda[(P_x, P_y)].$$

1. Adding Same Signs: $(-2) + (-1) = -3$

The base representations are

$$\text{Pair 1} = (P_0, \Sigma(\Sigma(P_0)))$$

and

$$\text{Pair 2} = (P_0, \Sigma(P_0)).$$

The expanded addition formula is

$$\Gamma = \Lambda \left[(\Sigma^{A(P_0, P_0)}(P_0), \Sigma^{A(\Sigma(\Sigma(P_0)), \Sigma(P_0))}(P_0)) \right].$$

First evaluate the positive component:

$$A(P_0, P_0) = \Theta \left(\Sigma^{\Theta(P_0)}(P_0) \right).$$

Since

$$\Theta(P_0) = 0,$$

we obtain

$$A(P_0, P_0) = \Theta(\Sigma^0(P_0)) = \Theta(P_0) = 0.$$

Thus

$$k_{\text{pos}} = 0.$$

Now evaluate the negative component:

$$A(\Sigma(\Sigma(P_0)), \Sigma(P_0)) = \Theta(\Sigma^{\Theta(\Sigma(P_0))}(\Sigma(\Sigma(P_0)))) .$$

Since

$$\Theta(\Sigma(P_0)) = 1,$$

we obtain

$$A(\Sigma(\Sigma(P_0)), \Sigma(P_0)) = \Theta(\Sigma^1(\Sigma(\Sigma(P_0)))) .$$

Therefore

$$A(\Sigma(\Sigma(P_0)), \Sigma(P_0)) = \Theta(\Sigma(\Sigma(\Sigma(P_0)))) = 3.$$

Thus

$$k_{\text{neg}} = 3.$$

Construct the new sets:

$$P_x = \Sigma^0(P_0) = P_0,$$

and

$$P_y = \Sigma^3(P_0) = \Sigma(\Sigma(\Sigma(P_0))).$$

Therefore

$$\Gamma = \Lambda[(P_0, \Sigma(\Sigma(\Sigma(P_0))))].$$

Since the first component is P_0 , reduction stops immediately:

$$\Lambda[(P_0, \Sigma(\Sigma(\Sigma(P_0))))] = (P_0, \Sigma(\Sigma(\Sigma(P_0)))).$$

But

$$\Sigma(\Sigma(\Sigma(P_0))) = P_3.$$

Hence the reduced pair is

$$(P_0, P_3).$$

By signed notation,

$$(P_0, P_3) \equiv -3.$$

Therefore

$$\boxed{(-2) + (-1) = -3.}$$

2. Adding Opposite Signs: $(-2) + 1 = -1$

The base representations are

$$\text{Pair 1} = (P_0, \Sigma(\Sigma(P_0)))$$

and

$$\text{Pair 2} = (\Sigma(P_0), P_0).$$

The expanded addition formula is

$$\Gamma = \Lambda \left[(\Sigma^{A(P_0, \Sigma(P_0))}(P_0), \Sigma^{A(\Sigma(\Sigma(P_0)), P_0)}(P_0)) \right].$$

Evaluate the positive component:

$$A(P_0, \Sigma(P_0)) = \Theta \left(\Sigma^{\Theta(\Sigma(P_0))}(P_0) \right).$$

Since

$$\Theta(\Sigma(P_0)) = 1,$$

we have

$$A(P_0, \Sigma(P_0)) = \Theta(\Sigma^1(P_0)) = \Theta(\Sigma(P_0)) = 1.$$

Thus

$$k_{\text{pos}} = 1.$$

Now evaluate the negative component:

$$A(\Sigma(\Sigma(P_0)), P_0) = \Theta(\Sigma^{\Theta(P_0)}(\Sigma(\Sigma(P_0)))) .$$

Since

$$\Theta(P_0) = 0,$$

we obtain

$$A(\Sigma(\Sigma(P_0)), P_0) = \Theta(\Sigma^0(\Sigma(\Sigma(P_0)))) = \Theta(\Sigma(\Sigma(P_0))) = 2.$$

Thus

$$k_{\text{neg}} = 2.$$

Construct the sets:

$$P_x = \Sigma^1(P_0) = \Sigma(P_0),$$

and

$$P_y = \Sigma^2(P_0) = \Sigma(\Sigma(P_0)).$$

Therefore

$$\Gamma = \Lambda[(\Sigma(P_0), \Sigma(\Sigma(P_0)))].$$

Because both sets are nonempty,

$$\Lambda[(\Sigma(P_0), \Sigma(\Sigma(P_0)))] = \Lambda[(N(\Sigma(P_0)), N(\Sigma(\Sigma(P_0))))].$$

By the definition of N ,

$$N(\Sigma(P_0)) = P_0$$

and

$$N(\Sigma(\Sigma(P_0))) = \Sigma(P_0).$$

Therefore

$$\Lambda[(\Sigma(P_0), \Sigma(\Sigma(P_0)))] = \Lambda[(P_0, \Sigma(P_0))].$$

The first component is now P_0 , so

$$\Lambda[(P_0, \Sigma(P_0))] = (P_0, \Sigma(P_0)).$$

Since

$$\Sigma(P_0) = P_1,$$

the reduced pair is

$$(P_0, P_1).$$

By signed notation,

$$(P_0, P_1) \equiv -1.$$

Therefore

$$\boxed{(-2) + 1 = -1.}$$

3. Subtracting Same Signs: $1 - 2 = -1$

The base representations are

$$\text{Pair 1} = (\Sigma(P_0), P_0)$$

and

$$\text{Pair 2} = (\Sigma(\Sigma(P_0)), P_0).$$

The expanded subtraction formula is

$$\Pi = \Lambda \left[\left(\Sigma^{A(\Sigma(P_0), P_0)}(P_0), \Sigma^{A(P_0, \Sigma(\Sigma(P_0)))}(P_0) \right) \right].$$

Evaluate the positive component:

$$A(\Sigma(P_0), P_0) = \Theta \left(\Sigma^{\Theta(P_0)}(\Sigma(P_0)) \right).$$

Since

$$\Theta(P_0) = 0,$$

we obtain

$$A(\Sigma(P_0), P_0) = \Theta(\Sigma^0(\Sigma(P_0))) = \Theta(\Sigma(P_0)) = 1.$$

Thus

$$k_{\text{pos}} = 1.$$

Evaluate the negative component:

$$A(P_0, \Sigma(\Sigma(P_0))) = \Theta \left(\Sigma^{\Theta(\Sigma(\Sigma(P_0)))}(P_0) \right).$$

Since

$$\Theta(\Sigma(\Sigma(P_0))) = 2,$$

we obtain

$$A(P_0, \Sigma(\Sigma(P_0))) = \Theta(\Sigma^2(P_0)) = \Theta(\Sigma(\Sigma(P_0))) = 2.$$

Thus

$$k_{\text{neg}} = 2.$$

Construct the sets:

$$P_x = \Sigma^1(P_0) = \Sigma(P_0),$$

and

$$P_y = \Sigma^2(P_0) = \Sigma(\Sigma(P_0)).$$

Therefore

$$\Pi = \Lambda[(\Sigma(P_0), \Sigma(\Sigma(P_0)))].$$

Reduction gives

$$\Lambda[(\Sigma(P_0), \Sigma(\Sigma(P_0)))] = \Lambda[(P_0, \Sigma(P_0))] = (P_0, \Sigma(P_0)).$$

Since

$$\Sigma(P_0) = P_1,$$

we obtain

$$(P_0, \Sigma(P_0)) = (P_0, P_1).$$

By signed notation,

$$(P_0, P_1) \equiv -1.$$

Therefore

$$\boxed{1 - 2 = -1.}$$

4. Subtracting Opposite Signs: $1 - (-2) = 3$

The base representations are

$$\text{Pair 1} = (\Sigma(P_0), P_0)$$

and

$$\text{Pair 2} = (P_0, \Sigma(\Sigma(P_0))).$$

The expanded subtraction formula is

$$\Pi = \Lambda \left[\left(\Sigma^{A(\Sigma(P_0), \Sigma(\Sigma(P_0)))}(P_0), \Sigma^{A(P_0, P_0)}(P_0) \right) \right].$$

Evaluate the positive component:

$$A(\Sigma(P_0), \Sigma(\Sigma(P_0))) = \Theta \left(\Sigma^{\Theta(\Sigma(\Sigma(P_0)))}(\Sigma(P_0)) \right).$$

Since

$$\Theta(\Sigma(\Sigma(P_0))) = 2,$$

we obtain

$$A(\Sigma(P_0), \Sigma(\Sigma(P_0))) = \Theta(\Sigma^2(\Sigma(P_0))).$$

Therefore

$$A(\Sigma(P_0), \Sigma(\Sigma(P_0))) = \Theta(\Sigma(\Sigma(\Sigma(P_0)))) = 3.$$

Thus

$$k_{\text{pos}} = 3.$$

Evaluate the negative component:

$$A(P_0, P_0) = \Theta(\Sigma^{\Theta(P_0)}(P_0)).$$

Since

$$\Theta(P_0) = 0,$$

we obtain

$$A(P_0, P_0) = \Theta(P_0) = 0.$$

Thus

$$k_{\text{neg}} = 0.$$

Construct the sets:

$$P_x = \Sigma^3(P_0) = \Sigma(\Sigma(\Sigma(P_0))),$$

and

$$P_y = \Sigma^0(P_0) = P_0.$$

Therefore

$$\Pi = \Lambda[(\Sigma(\Sigma(\Sigma(P_0))), P_0)].$$

Since the second component is already P_0 ,

$$\Lambda[(\Sigma(\Sigma(\Sigma(P_0))), P_0)] = (\Sigma(\Sigma(\Sigma(P_0))), P_0).$$

Since

$$\Sigma(\Sigma(\Sigma(P_0))) = P_3,$$

the reduced pair is

$$(P_3, P_0).$$

By signed notation,

$$(P_3, P_0) \equiv 3.$$

Therefore

$$\boxed{1 - (-2) = 3.}$$

Non-Unique Signed-Pair Representations

The signed-pair construction admits infinitely many unreduced representations of the same signed integer.

Let

$$(P_a, P_b)$$

be any signed-pair representation, and let

$$P_v \in \mathcal{P}.$$

Construct the shifted pair

$$(\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b)).$$

Since

$$\Theta(P_v) = v,$$

and applying Σ exactly v times to a canonical set P_a produces P_{a+v} , we have

$$\Sigma^{\Theta(P_v)}(P_a) = P_{a+v}$$

and

$$\Sigma^{\Theta(P_v)}(P_b) = P_{b+v}.$$

Hence the shifted pair may be written

$$\boxed{(\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b)) = (P_{a+v}, P_{b+v}).}$$

Invariance Under Equal Shifting

Theorem 2. *For every $P_a, P_b, P_v \in \mathcal{P}$,*

$$\boxed{\Lambda[(P_a, P_b)] = \Lambda[(P_{a+v}, P_{b+v})].}$$

Equivalently,

$$\boxed{\Lambda[(P_a, P_b)] = \Lambda[(\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b))].}$$

Proof. The pair

$$(P_{a+v}, P_{b+v})$$

contains the same additional contribution v in both components.

By the definition of Λ , whenever both components are nonempty, one application of N is made to each component simultaneously.

Applying this simultaneous removal v times gives

$$\Lambda[(P_{a+v}, P_{b+v})] = \Lambda[(P_a, P_b)].$$

Therefore adding the same canonical contribution to both components does not change the reduced signed integer represented by the pair.

Hence

$$\boxed{\Lambda[(P_{a+v}, P_{b+v})] = \Lambda[(P_a, P_b)].}$$

□

Thus a single signed integer does not have only one possible unreduced representation.
For example,

$$(P_a, P_b),$$

$$(P_{a+1}, P_{b+1}),$$

$$(P_{a+2}, P_{b+2}),$$

and, more generally,

$$(P_{a+v}, P_{b+v})$$

all reduce to the same signed integer.

Because the canonical family

$$\mathcal{P} = \{P_0, P_1, P_2, P_3, \dots\}$$

contains infinitely many canonical sets, there are infinitely many possible choices of P_v , and therefore infinitely many unreduced representations of the same signed integer.

Non-Unique Input Representations for Signed Addition

Consider

$$\Gamma((P_a, P_b), (P_c, P_d)).$$

Let

$$P_v, P_w \in \mathcal{P}.$$

Independently shift the two input pairs by P_v and P_w , respectively.

The first pair becomes

$$(\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b)),$$

and the second becomes

$$(\Sigma^{\Theta(P_w)}(P_c), \Sigma^{\Theta(P_w)}(P_d)).$$

Therefore,

$$\begin{aligned} & \Gamma((P_a, P_b), (P_c, P_d)) \\ &= \Gamma((\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b)), (\Sigma^{\Theta(P_w)}(P_c), \Sigma^{\Theta(P_w)}(P_d))). \end{aligned}$$

Using the canonical-set notation, this is equivalently

$$\Gamma((P_a, P_b), (P_c, P_d)) = \Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w})).$$

Theorem 3. *For every*

$$P_a, P_b, P_c, P_d, P_v, P_w \in \mathcal{P},$$

the equality

$$\Gamma((P_a, P_b), (P_c, P_d)) = \Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w}))$$

holds.

Proof. By the preceding theorem,

$$\Lambda[(P_{a+v}, P_{b+v})] = \Lambda[(P_a, P_b)]$$

and independently,

$$\Lambda[(P_{c+w}, P_{d+w})] = \Lambda[(P_c, P_d)].$$

Thus each shifted pair represents exactly the same signed integer as its corresponding original pair.

The definition of signed addition combines the positive components, combines the negative components, and then applies Λ .

Adding v to both components of the first pair contributes the same additional amount to the positive and negative sides.

Likewise, adding w to both components of the second pair contributes the same additional amount to the positive and negative sides.

These equal contributions cancel under Λ .

Therefore

$$\Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w})) = \Gamma((P_a, P_b), (P_c, P_d)).$$

□

Infinitely Many Addition Representations

Since P_v and P_w may each be chosen independently from the infinite family

$$\mathcal{P},$$

a single signed addition may be represented by infinitely many different input tuples.
Thus

$$\Gamma((P_a, P_b), (P_c, P_d))$$

may equivalently be evaluated using

$$\Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w}))$$

for arbitrary

$$v, w \in \mathbb{N}.$$

For example, all of the following produce the same signed output:

$$\Gamma((P_a, P_b), (P_c, P_d)),$$

$$\Gamma((P_{a+1}, P_{b+1}), (P_{c+1}, P_{d+1})),$$

$$\Gamma((P_{a+5}, P_{b+5}), (P_{c+9}, P_{d+9})),$$

and

$$\Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w}))$$

for arbitrary v and w .

Consequently, the mapping from such input representations to the reduced signed output is not injective.

That is, if only the final reduced output

$$P_{\text{out}} = \Gamma((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w}))$$

is known, the output alone does not uniquely determine the values of

$$v \quad \text{and} \quad w.$$

Nor does it uniquely determine the particular unreduced input pairs used in the computation. There are infinitely many distinct admissible input representations that may correspond to the same signed output.

This is a many-to-one property of the representation system.

It should not by itself be interpreted as a proof of computational hardness or as a statement concerning the complexity-theoretic question P versus NP . Rather, it establishes that information about the particular input representation is lost when the inputs are reduced to a single signed result.

Non-Unique Input Representations for Signed Subtraction

The same construction applies to signed subtraction.

Recall that

$$\Pi((P_a, P_b), (P_c, P_d))$$

is defined by adding the additive inverse of the second pair.

Let

$$P_v, P_w \in \mathcal{P}.$$

Then

$$(P_a, P_b)$$

and

$$(P_{a+v}, P_{b+v})$$

represent the same signed integer, while

$$(P_c, P_d)$$

and

$$(P_{c+w}, P_{d+w})$$

also represent the same signed integer.

Therefore,

$$\boxed{\Pi((P_a, P_b), (P_c, P_d)) = \Pi((P_{a+v}, P_{b+v}), (P_{c+w}, P_{d+w})) .}$$

Equivalently,

$$\boxed{\begin{aligned} &\Pi((P_a, P_b), (P_c, P_d)) \\ &= \Pi((\Sigma^{\Theta(P_v)}(P_a), \Sigma^{\Theta(P_v)}(P_b)), (\Sigma^{\Theta(P_w)}(P_c), \Sigma^{\Theta(P_w)}(P_d))) . \end{aligned}}$$

Hence signed subtraction also admits infinitely many distinct unreduced input representations that produce the same reduced signed output.

Input-to-Output Representation Expansion

A standard signed addition

$$\Gamma((P_a, P_b), (P_c, P_d))$$

requires four canonical input sets:

$$P_a, \quad P_b, \quad P_c, \quad P_d.$$

Under the non-unique representation construction, two additional canonical sets may be chosen:

$$P_v \quad \text{and} \quad P_w.$$

These determine the independent shifts applied to the two signed input pairs.

The complete representation may therefore depend upon the six canonical sets

$$\boxed{P_a, P_b, P_c, P_d, P_v, P_w.}$$

while the function ultimately returns a single signed-integer value, represented canonically by one reduced signed pair.

If one merely counts the six selected canonical-set parameters against the one resulting signed value, the representation has the bookkeeping description

$$\boxed{6 : 1}$$

of input-set parameters to output value:

$$\boxed{6 \text{ selected canonical-set parameters} \longrightarrow 1 \text{ signed output value.}}$$

This is not a statement that the function has exactly six possible inputs or six preimages for each output. The preceding results show the opposite: infinitely many distinct shifted input representations can produce the same signed output.

The sets P_v and P_w are optional in the sense that choosing

$$P_v = P_0 \quad \text{and} \quad P_w = P_0$$

returns the original unshifted representation.

Therefore the ordinary signed operation is contained as the special case

$$v = w = 0.$$

The additional sets P_v and P_w do not alter the resulting signed value. They alter only the particular unreduced representation used to obtain that value.

This distinction between the represented value and the particular representation used to encode that value will be relevant when considering later applications involving concealed or randomized representations.

Defining Multiplication

We now define multiplication on signed integers using the signed-pair representation already constructed.

Let

$$(P_a, P_b)$$

and

$$(P_c, P_d)$$

be signed-pair representations.

First reduce both pairs:

$$\Lambda[(P_a, P_b)]$$

and

$$\Lambda[(P_c, P_d)].$$

Let the signed output symbols represented by these reduced pairs be

$$b$$

and

$$c,$$

respectively. Thus,

$$\Lambda[(P_a, P_b)] \equiv b$$

and

$$\Lambda[(P_c, P_d)] \equiv c.$$

The multiplication function for the two signed-pair inputs will be defined after multiplication of the signed symbols b and c has been constructed below.

1. Absolute-Value Function

We first define a function

$$\text{abs}$$

on signed output symbols.

For any signed output symbol δ , define

$\text{abs}(\delta) = \begin{cases} \delta, & \delta \text{ is nonnegative,} \\ \text{the corresponding nonnegative symbol obtained by removing the sign } -, & \delta \text{ is negative.} \end{cases}$
--

Thus,

$$\text{abs}(0) = 0,$$

$$\text{abs}(1) = 1,$$

$$\text{abs}(2) = 2,$$

and

$$\text{abs}(-1) = 1,$$

$$\text{abs}(-2) = 2,$$

and so on.

Hence

$$\text{abs}(\delta)$$

always produces a nonnegative natural-number symbol.

2. Repeated Blocks of Successor Applications

We now introduce the notation

$$\Sigma_v^u(P)$$

for nonnegative natural-number symbols

$$u, v \in \mathbb{N}.$$

The expression

$$\Sigma_v^u(P)$$

means:

apply Σ to the input P exactly u times per block, and repeat that block exactly v times.

This notation is defined recursively and therefore does not assume multiplication.

For every canonical set P , define the base case

$$\boxed{\Sigma_0^u(P) = P.}$$

Thus zero blocks of u successor applications leave the input unchanged.

For the recursive step, suppose

$$\Sigma_v^u(P)$$

has already been constructed.

Then define

$$\boxed{\Sigma_{v+1}^u(P) = \Sigma^u(\Sigma_v^u(P)) .}$$

Here

$$\Sigma^u(Q)$$

means the already-defined operation of applying Σ to Q exactly u times.

Therefore each increase of the lower index by one applies one additional block of u successor operations.

For example,

$$\Sigma_0^3(P_0) = P_0.$$

Then

$$\Sigma_1^3(P_0) = \Sigma^3(P_0),$$

and

$$\Sigma_2^3(P_0) = \Sigma^3(\Sigma_1^3(P_0)) .$$

Substituting the definition of $\Sigma_1^3(P_0)$,

$$\Sigma_2^3(P_0) = \Sigma^3(\Sigma^3(P_0)) .$$

Expanding the first block,

$$\Sigma^3(P_0) = \Sigma(\Sigma(\Sigma(P_0))) .$$

Therefore,

$$\Sigma_2^3(P_0) = \Sigma^3(\Sigma(\Sigma(\Sigma(P_0)))) .$$

Expanding the second block gives

$$\Sigma_2^3(P_0) = \Sigma(\Sigma(\Sigma(\Sigma(\Sigma(\Sigma(P_0)))))).$$

Hence

$$\Theta(\Sigma_2^3(P_0)) = 6.$$

The value 6 is therefore produced from repeated successor operations alone, without assuming multiplication beforehand.

More generally,

$$\Sigma_v^u(P_0)$$

constructs the canonical set obtained by applying Σ in v consecutive blocks, each containing exactly u applications.

3. Unsigned Multiplication

Let

$$b, c$$

be the signed output symbols obtained from

$$\Lambda[(P_a, P_b)]$$

and

$$\Lambda[(P_c, P_d)].$$

Since

$$\text{abs}(b)$$

and

$$\text{abs}(c)$$

are both nonnegative natural-number symbols, the expression

$$\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0)$$

is well-defined.

We define the unsigned magnitude of the multiplication result by

$$M(b, c) = \Theta \left(\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0) \right).$$

Thus $M(b, c)$ is always a nonnegative natural-number symbol.

For example, if

$$b = 3$$

and

$$c = 2,$$

then

$$M(3, 2) = \Theta \left(\Sigma_2^3(P_0) \right).$$

From the recursive construction above,

$$\Sigma_2^3(P_0) = \Sigma(\Sigma(\Sigma(\Sigma(\Sigma(\Sigma(P_0)))))),$$

so

$$M(3, 2) = 6.$$

Likewise, if

$$b = -3$$

and

$$c = 2,$$

then

$$\text{abs}(-3) = 3$$

and

$$\text{abs}(2) = 2,$$

so the unsigned magnitude is still

$$M(-3, 2) = \Theta(\Sigma_2^3(P_0)) = 6.$$

The sign of the final multiplication result is determined separately.

4. Sign of the Multiplication Result

The sign of the result depends on the signs of

$$b$$

and

$$c.$$

If either input is zero, define

$$\Upsilon(b, c) = 0$$

whenever

$$b = 0$$

or

$$c = 0.$$

If b and c are both positive, the result is nonnegative:

$$\Upsilon(b, c) = M(b, c).$$

If b and c are both negative, the result is also nonnegative:

$$\Upsilon(b, c) = M(b, c).$$

If exactly one of b and c is negative, the result is the corresponding negative output symbol:

$$\Upsilon(b, c) = -M(b, c).$$

Therefore,

$$\Upsilon(b, c) = \begin{cases} 0, & b = 0 \text{ or } c = 0, \\ M(b, c), & b, c \text{ have the same sign,} \\ -M(b, c), & b, c \text{ have different signs.} \end{cases}$$

Substituting the definition of $M(b, c)$,

$$\Upsilon(b, c) = \begin{cases} 0, & b = 0 \text{ or } c = 0, \\ \Theta \left(\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0) \right), & b, c \text{ have the same sign,} \\ -\Theta \left(\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0) \right), & b, c \text{ have different signs.} \end{cases}$$

Once $\Upsilon(b, c)$ has been defined, we introduce the ordinary multiplication notation

$$b \cdot c := \Upsilon(b, c).$$

This notation does not introduce a new operation; it is only a shorter name for the multiplication function already constructed.

5. Multiplication of Signed-Pair Representations

Returning to the original signed-pair inputs,

$$(P_a, P_b)$$

and

$$(P_c, P_d),$$

let

$$\Lambda[(P_a, P_b)] \equiv b$$

and

$$\Lambda[(P_c, P_d)] \equiv c.$$

Then define

$$\Upsilon [(P_a, P_b), (P_c, P_d)] = \Upsilon(b, c).$$

Equivalently, reduce both input pairs first, identify the signed symbols represented by those reduced pairs as b and c , and then evaluate $\Upsilon(b, c)$. The reduction function itself still returns a reduced ordered pair; the symbols b and c are the signed notations assigned to those reduced pairs.

Expanding the definition completely,

$$\Upsilon [(P_a, P_b), (P_c, P_d)] = \begin{cases} 0, & b = 0 \text{ or } c = 0, \\ \Theta \left(\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0) \right), & b, c \text{ have the same sign,} \\ -\Theta \left(\Sigma_{\text{abs}(c)}^{\text{abs}(b)}(P_0) \right), & b, c \text{ have different signs,} \end{cases}$$

where

$$\Lambda[(P_a, P_b)] \equiv b$$

and

$$\Lambda[(P_c, P_d)] \equiv c.$$

Thus multiplication is constructed entirely from the previously defined operations

$$P_0, \quad \Sigma, \quad \Theta, \quad \Lambda,$$

together with the recursively defined repeated-block notation

$$\Sigma_v^u(P),$$

which is instantiated in multiplication with

$$u = \text{abs}(b), \quad v = \text{abs}(c).$$

Euclidean Division and Modular Remainders

We now define a division-and-remainder procedure using the previously constructed signed integers, absolute value, successor operation, and deterministic element removal.

Let

$$(P_a, P_b)$$

and

$$(P_c, P_d)$$

be signed-pair representations.

Reduce both pairs and let the signed symbols represented by the reduced pairs be

$$b$$

and

$$c,$$

respectively. Thus

$$\Lambda[(P_a, P_b)] \equiv b$$

and

$$\Lambda[(P_c, P_d)] \equiv c.$$

The divisor must be nonzero, so throughout this section assume

$$\boxed{c \neq 0.}$$

As in the multiplication section, b and c here denote signed output symbols represented by reduced pairs.

1. Magnitude Sets

Construct the canonical set representing the magnitude of the dividend:

$$Q_0 = \Sigma^{\text{abs}(b)}(P_0).$$

Construct the canonical set representing the magnitude of the divisor:

$$C = \Sigma^{\text{abs}(c)}(P_0).$$

Therefore

$$\Theta(Q_0) = \text{abs}(b)$$

and

$$\Theta(C) = \text{abs}(c).$$

Also initialize an empty output-count set

$$O_0 = P_0.$$

The set Q_n will record the amount still remaining after n complete divisor-sized removals, while O_n will record how many complete removals have occurred.

2. Full-Block Removal Rule

A divisor-sized block may be removed only in full. No partial block is permitted.

Suppose Q_n and O_n have already been constructed. If

$$\Theta(C) \leq \Theta(Q_n),$$

then define

$$Q_{n+1} = N^{\Theta(C)}(Q_n)$$

and simultaneously define

$$O_{n+1} = \Sigma(O_n).$$

Thus one complete block containing exactly $\Theta(C)$ elements is removed from Q_n , and exactly one successor is applied to the count set O_n .

Whenever such a full block is permitted, the previously defined natural-subtraction function records the same numerical step:

$$\Delta(Q_n, C) = \Theta(N^{\Theta(C)}(Q_n)) = \Theta(Q_{n+1}).$$

If instead

$$\Theta(Q_n) < \Theta(C),$$

then no removal is performed and the process stops.

This rule implements the intended repeated natural subtraction without creating a type inconsistency. The previously defined function $\Delta(P_1, P_2)$ returns an output symbol, not a set, so it cannot itself be iterated as a set transformation. The actual repeated removal underlying Δ is therefore written using $N^{\Theta(C)}$.

Because each successful step removes at least one element from the finite set Q_n , and because $c \neq 0$ implies $\Theta(C) > 0$, the process terminates after finitely many steps.

Let m be the first stage at which

$$\Theta(Q_m) < \Theta(C).$$

Define the unsigned quotient magnitude

$$\boxed{q_0 := \Theta(Q_m)}$$

and the unsigned leftover magnitude

$$\boxed{r_0 := \Theta(Q_m)}.$$

By construction,

$$0 \leq r_0 < \text{abs}(c).$$

For example, if

$$b = 17, \quad c = 5,$$

then the successive magnitudes of Q_n are

$$17, 12, 7, 2.$$

Three complete blocks are removed, so

$$q_0 = 3, \quad r_0 = 2.$$

3. Signed Euclidean Quotient and Remainder

The unsigned process above divides $\text{abs}(b)$ by $\text{abs}(c)$.

We now assign signs so that the final quotient q and remainder r always satisfy

$$\boxed{b = q \cdot c + r}$$

and

$$\boxed{0 \leq r < \text{abs}(c).}$$

If b is nonnegative, define

$$r = r_0,$$

and define

$$q = \begin{cases} q_0, & c > 0, \\ -q_0, & c < 0. \end{cases}$$

If b is negative and $r_0 = 0$, define

$$r = 0,$$

and

$$q = \begin{cases} -q_0, & c > 0, \\ q_0, & c < 0. \end{cases}$$

Finally, if b is negative and $r_0 > 0$, one additional divisor-sized block is required in the signed quotient so that the remainder remains nonnegative.

Define

$$\boxed{r = \text{abs}(c) - r_0}$$

and

$$\boxed{q = \begin{cases} -(q_0 + 1), & c > 0, \\ q_0 + 1, & c < 0. \end{cases}}$$

This adjustment is necessary because simply attaching a negative sign to q_0 would produce a negative remainder when the dividend is negative and $r_0 \neq 0$.

4. Definition of Ξ

Define

$$\boxed{\Xi[(P_a, P_b), (P_c, P_d)] = \Xi(b, c) = (q, r),}$$

where b and c are the signed symbols represented by

$$\Lambda[(P_a, P_b)]$$

and

$$\Lambda[(P_c, P_d)],$$

and where q and r are constructed by the full-block removal and sign rules above.

Thus Ξ returns two output symbols:

$$\boxed{\text{quotient } q \quad \text{and} \quad \text{remainder } r.}$$

The output always satisfies

$$\boxed{b = q \cdot c + r, \quad 0 \leq r < \text{abs}(c).}$$

5. Examples

For $b = 17$ and $c = 5$,

$$\Xi(17, 5) = (3, 2),$$

because

$$17 = 3 \cdot 5 + 2.$$

For $b = -17$ and $c = 5$, the unsigned process still gives

$$q_0 = 3, \quad r_0 = 2.$$

Since the dividend is negative and $r_0 > 0$,

$$q = -(3 + 1) = -4$$

and

$$r = 5 - 2 = 3.$$

Hence

$$\Xi(-17, 5) = (-4, 3),$$

and indeed

$$-17 = (-4) \cdot 5 + 3.$$

For $b = 17$ and $c = -5$,

$$\Xi(17, -5) = (-3, 2),$$

because

$$17 = (-3) \cdot (-5) + 2.$$

For $b = -17$ and $c = -5$,

$$\Xi(-17, -5) = (4, 3),$$

because

$$-17 = 4 \cdot (-5) + 3.$$

6. Modulo

For modular arithmetic, the modulus is taken to be a positive natural-number symbol m .

If

$$\Xi(b, m) = (q, r),$$

then define

$$\boxed{b \bmod m := r.}$$

Thus the modulo operation returns the nonnegative remainder produced by Ξ , and it always satisfies

$$\boxed{0 \leq b \bmod m < m.}$$

For example,

$$17 \bmod 5 = 2$$

and

$$-17 \bmod 5 = 3.$$

Two signed integers x and y are said to be congruent modulo m when they produce the same remainder under this operation.

We write

$$\boxed{x \equiv y \pmod{m}}$$

precisely when

$$x \bmod m = y \bmod m.$$

For example,

$$17 \equiv 2 \pmod{5}$$

and

$$-17 \equiv 3 \pmod{5}.$$

Defining Quotient and Remainder Output Functions

Recall that for signed integers b and $c \neq 0$,

$$\Xi(b, c) = (q, r),$$

where

$$b = q \cdot c + r$$

and

$$0 \leq r < \text{abs}(c).$$

We now define two functions that extract the two outputs of Ξ .

1. Quotient Output Function

If

$$\Xi(b, c) = (q, r),$$

define

$$\boxed{\text{quo}(\Xi(b, c)) := q.}$$

Therefore,

$$\text{quo}(\Xi(b, c))$$

returns only the quotient produced by Ξ .

For signed-pair inputs,

$$\boxed{\text{quo}(\Xi[(P_a, P_b), (P_c, P_d)]) := q}$$

whenever

$$\Xi[(P_a, P_b), (P_c, P_d)] = (q, r).$$

2. Remainder Output Function

If

$$\Xi(b, c) = (q, r),$$

define

$$\boxed{\text{rem}(\Xi(b, c)) := r.}$$

Therefore,

$$\text{rem}(\Xi(b, c))$$

returns only the remainder produced by Ξ .

For signed-pair inputs,

$$\boxed{\text{rem}(\Xi[(P_a, P_b), (P_c, P_d)]) := r}$$

whenever

$$\Xi[(P_a, P_b), (P_c, P_d)] = (q, r).$$

Because Ξ is defined so that

$$0 \leq r < \text{abs}(c),$$

the remainder function always satisfies

$$\boxed{0 \leq \text{rem}(\Xi(b, c)) < \text{abs}(c).}$$

When the divisor is a positive modulus v ,

$$c = v > 0,$$

this becomes

$$\boxed{0 \leq \text{rem}(\Xi(b, v)) < v.}$$

Defining Vectors over $\mathbb{Z}_v$

We now define finite vectors whose coordinates are signed integers reduced modulo a common positive modulus v .

Let

$$u \in \mathbb{N}$$

be the number of coordinates in the vector, and let

$$v \in \mathbb{N}$$

with

$$v > 0.$$

A vector containing exactly u coordinates modulo v is written

$$\boxed{\vec{x} \in \mathbb{Z}_v^u.}$$

Here,

u

specifies the number of coordinates, while

v

specifies the modulus used for every coordinate.

1. Construction of a Coordinate

Each coordinate is constructed from two signed-pair representations.

Let

$$(P_a, P_b)$$

be any signed-pair representation.

Its reduced signed value may be any integer:

$$\Lambda[(P_a, P_b)] \equiv s.$$

Let

$$(P_c, P_d)$$

be a signed-pair representation satisfying

$$\boxed{\Lambda[(P_c, P_d)] \equiv v,}$$

where

$$v > 0.$$

Then

$$\Xi[(P_a, P_b), (P_c, P_d)]$$

produces a quotient and remainder.

The coordinate stored in the vector is defined to be only the remainder:

$$\boxed{x_1 = \text{rem}(\Xi[(P_a, P_b), (P_c, P_d)]) .}$$

Therefore

$$0 \leq x_1 < v.$$

The first signed pair may represent any signed integer, while the second signed pair represents the fixed positive modulus v .

2. Construction of the Vector

Let there be exactly u signed-pair inputs

$$(P_{a_1}, P_{b_1}), (P_{a_2}, P_{b_2}), \dots, (P_{a_u}, P_{b_u}),$$

and exactly u modulus-pair inputs

$$(P_{c_1}, P_{d_1}), (P_{c_2}, P_{d_2}), \dots, (P_{c_u}, P_{d_u}),$$

where every modulus pair represents the same positive signed value v :

$$\boxed{\Lambda[(P_{c_i}, P_{d_i})] \equiv v}$$

for every

$$i = 1, 2, \dots, u.$$

Define each coordinate by

$$\boxed{x_i = \text{rem}(\Xi[(P_{a_i}, P_{b_i}), (P_{c_i}, P_{d_i})]) .}$$

Then define the vector

$$\boxed{\vec{x} = (x_1, x_2, \dots, x_u) .}$$

Substituting the definitions of the coordinates,

$$\boxed{\vec{x} = (\text{rem}(\Xi[(P_{a_1}, P_{b_1}), (P_{c_1}, P_{d_1})]), \text{rem}(\Xi[(P_{a_2}, P_{b_2}), (P_{c_2}, P_{d_2})]), \dots, \text{rem}(\Xi[(P_{a_u}, P_{b_u}), (P_{c_u}, P_{d_u})])) .}$$

Since every coordinate satisfies

$$0 \leq x_i < v,$$

the resulting vector satisfies

$$\boxed{\vec{x} \in \mathbb{Z}_v^u.}$$

3. Expanded Three-Coordinate Example

For a vector with

$$u = 3,$$

write

$$\vec{x} \in \mathbb{Z}_v^3.$$

Let

$$(P_a, P_b), \quad (P_e, P_f), \quad (P_i, P_j)$$

be three arbitrary signed-pair representations.

Let

$$(P_c, P_d), \quad (P_g, P_h), \quad (P_k, P_l)$$

all represent the same positive modulus v :

$$\Lambda[(P_c, P_d)] \equiv v,$$

$$\Lambda[(P_g, P_h)] \equiv v,$$

and

$$\Lambda[(P_k, P_l)] \equiv v.$$

Then define

$$\boxed{\vec{x} = (\text{rem}(\Xi[(P_a, P_b), (P_c, P_d)]), \text{rem}(\Xi[(P_e, P_f), (P_g, P_h)]), \text{rem}(\Xi[(P_i, P_j), (P_k, P_l)])) .}$$

Each first ordered pair may represent any signed integer.

Each second ordered pair must represent the same positive modulus v .

Therefore each coordinate lies in

$$\{0, 1, 2, \dots, v - 1\},$$

and the complete vector contains exactly three coordinates.

Hence

$$\vec{x} \in \mathbb{Z}_v^3.$$

4. General Form

More generally,

$$\vec{x} \in \mathbb{Z}_v^u.$$

means that

$$\vec{x} = (x_1, x_2, \dots, x_u)$$

contains exactly u coordinates and every coordinate satisfies

$$0 \leq x_i < v.$$

Within the present system, each coordinate is obtained from

$$x_i = \text{rem}(\Xi[(P_{a_i}, P_{b_i}), (P_{c_i}, P_{d_i})]),$$

with

$$\Lambda[(P_{c_i}, P_{d_i})] \equiv v$$

for every coordinate.

Thus the first signed-pair input may vary independently from coordinate to coordinate, while every second signed-pair input represents the same positive modulus v .

Matrix-Vector Multiplication with Signed-Pair Outputs

We now define a matrix, a vector, a canonical symbol-to-pair map, and the corresponding matrix-vector product.

1. Matrix Definition

Let

$$A = \begin{pmatrix} (P_{a_1}, P_{b_1}) & (P_{c_1}, P_{d_1}) \\ (P_{a_2}, P_{b_2}) & (P_{c_2}, P_{d_2}) \end{pmatrix}.$$

Each matrix entry is a signed-pair representation.

Thus the four entries represent signed integers through reduction by Λ :

$$\Lambda[(P_{a_1}, P_{b_1})], \quad \Lambda[(P_{c_1}, P_{d_1})],$$

$$\Lambda[(P_{a_2}, P_{b_2})], \quad \Lambda[(P_{c_2}, P_{d_2})].$$

No requirement is made that the matrix entries already be reduced.

2. Vector Definition

Let

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

The coordinates

$$x_1 \quad \text{and} \quad x_2$$

are signed numerical symbols.

Unlike the entries of A , the coordinates of $\vec{x}$ are not themselves ordered pairs of canonical sets.

Therefore, before a vector coordinate can be used as an input to Υ , it must be converted into a signed-pair representation.

3. Definition of Λ^{-1}

Define the canonical symbol-to-pair function

$$\Lambda^{-1}$$

on signed numerical symbols.

For any signed symbol s , define

$$\Lambda^{-1}(s) = \begin{cases} (P_{\text{abs}(s)}, P_0), & s > 0, \\ (P_0, P_{\text{abs}(s)}), & s < 0, \\ (P_0, P_0), & s = 0. \end{cases}$$

Equivalently, using the successor construction,

$$\Lambda^{-1}(s) = \begin{cases} (\Sigma^{\text{abs}(s)}(P_0), P_0), & s > 0, \\ (P_0, \Sigma^{\text{abs}(s)}(P_0)), & s < 0, \\ (P_0, P_0), & s = 0. \end{cases}$$

Thus, for example,

$$\Lambda^{-1}(3) = (P_3, P_0),$$

$$\Lambda^{-1}(-3) = (P_0, P_3),$$

and

$$\Lambda^{-1}(0) = (P_0, P_0).$$

4. Why Λ^{-1} Is Not a True Inverse

The notation

$$\Lambda^{-1}$$

is retained by definition, but it is not a true inverse function in the ordinary set-theoretic sense.

The reason is that infinitely many different unreduced signed pairs may represent the same signed numerical symbol.

For example,

$$(P_3, P_0),$$

$$(P_4, P_1),$$

$$(P_5, P_2),$$

and, more generally,

$$(P_{3+v}, P_v)$$

for arbitrary

$$v \in \mathbb{N}$$

all reduce to the same signed value.

Therefore, given only the symbol

$$3,$$

there is no unique original ordered pair that can be recovered.

The function

$$\Lambda^{-1}$$

does not attempt to recover the original unreduced representation.

Instead, it returns the unique canonical reduced pair corresponding to the signed symbol.

Hence,

$$\boxed{\Lambda^{-1}(3) = (P_3, P_0)}$$

even if the value 3 originally arose from

$$(P_8, P_5).$$

The function therefore satisfies the canonical recovery property

$$\boxed{\Lambda[\Lambda^{-1}(s)] \equiv s.}$$

However, in general,

$$\boxed{\Lambda^{-1}(\Lambda[(P_a, P_b)]) \neq (P_a, P_b)}$$

whenever the original pair is unreduced.

Instead,

$$\Lambda^{-1}(\Lambda[(P_a, P_b)])$$

returns the canonical reduced pair representing the same signed value.

5. Unreduced Matrix-Entry–Vector-Coordinate Multiplication

Since Υ accepts signed-pair inputs, a vector coordinate must first be converted using Λ^{-1} .

For the matrix entry

$$(P_{a_1}, P_{b_1})$$

and vector coordinate

$$x_1,$$

the product is

$$\Upsilon [(P_{a_1}, P_{b_1}), \Lambda^{-1}(x_1)] .$$

The output of Υ is a signed numerical symbol.

Since Γ requires signed-pair inputs, whenever this product is to be used as an input to Γ , the output of Υ must be converted back into a signed pair using Λ^{-1} .

Thus the corresponding signed-pair product is

$$\boxed{\Lambda^{-1}(\Upsilon [(P_{a_1}, P_{b_1}), \Lambda^{-1}(x_1)])} .$$

Likewise,

$$\boxed{\Lambda^{-1}(\Upsilon [(P_{c_1}, P_{d_1}), \Lambda^{-1}(x_2)])} .$$

6. Overbar Notation for Pre-Modular Results

From this point onward, an overbar placed over a vector expression denotes that the indicated arithmetic operation has been performed, but coordinate-wise modular reduction by

$$\text{rem}(\Xi(\cdot))$$

has not yet been applied.

Thus,

$$\boxed{\overline{\delta \vec{x}}, \quad \overline{\vec{x} + \vec{y}}, \quad \overline{A \vec{x}}}$$

denote pre-modular results.

The overbar does not specify whether the coordinates are signed numerical symbols or signed pairs. Their type is determined by the function producing them.

In particular:

$$\Upsilon$$

produces signed numerical symbols, while

$$\Gamma$$

produces reduced signed pairs.

The overbar indicates only that the final modular-remainder step has not yet been applied.

7. Matrix-Vector Multiplication Before Modular Reduction

Let

$$A = \begin{pmatrix} (P_{a_1}, P_{b_1}) & (P_{c_1}, P_{d_1}) \\ (P_{a_2}, P_{b_2}) & (P_{c_2}, P_{d_2}) \end{pmatrix}$$

and

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

The first coordinate of the pre-modular matrix-vector product

$$\overline{A \vec{x}}$$

is obtained by multiplying each entry of the first row by the corresponding vector coordinate and then combining the resulting signed-pair products using Γ .

Define

$$\boxed{\overline{y_1} = \Gamma \left(\Lambda^{-1} \left(\Upsilon \left[(P_{a_1}, P_{b_1}), \Lambda^{-1}(x_1) \right] \right), \Lambda^{-1} \left(\Upsilon \left[(P_{c_1}, P_{d_1}), \Lambda^{-1}(x_2) \right] \right) \right) .}$$

Similarly,

$$\overline{y}_2 = \Gamma \left(\Lambda^{-1} \left(\Upsilon \left[(P_{a_2}, P_{b_2}), \Lambda^{-1}(x_1) \right] \right), \Lambda^{-1} \left(\Upsilon \left[(P_{c_2}, P_{d_2}), \Lambda^{-1}(x_2) \right] \right) \right).$$

Therefore define

$$\overline{A\vec{x}} = \begin{pmatrix} \overline{y}_1 \\ \overline{y}_2 \end{pmatrix}.$$

Substituting the definitions directly,

$$\overline{A\vec{x}} = \begin{pmatrix} \Gamma \left(\Lambda^{-1} \left(\Upsilon \left[(P_{a_1}, P_{b_1}), \Lambda^{-1}(x_1) \right] \right), \Lambda^{-1} \left(\Upsilon \left[(P_{c_1}, P_{d_1}), \Lambda^{-1}(x_2) \right] \right) \right) \\ \Gamma \left(\Lambda^{-1} \left(\Upsilon \left[(P_{a_2}, P_{b_2}), \Lambda^{-1}(x_1) \right] \right), \Lambda^{-1} \left(\Upsilon \left[(P_{c_2}, P_{d_2}), \Lambda^{-1}(x_2) \right] \right) \right) \end{pmatrix}.$$

Each coordinate of

$$\overline{A\vec{x}}$$

is therefore a reduced signed-pair output produced by Γ .

At this stage,

$$\text{rem}(\Xi(\cdot))$$

has not been applied.

Thus

$$\overline{A\vec{x}}$$

denotes the ordinary signed matrix-vector product before reduction modulo a chosen modulus.

The type flow of each individual multiplication term is

$$\text{symbol} \xrightarrow{\Lambda^{-1}} \text{signed pair} \xrightarrow{\Upsilon} \text{signed symbol} \xrightarrow{\Lambda^{-1}} \text{signed pair}.$$

The resulting signed-pair products are then combined using Γ .

Vector Operations

Let

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_u \end{pmatrix}$$

and

$$\vec{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_u \end{pmatrix}$$

be vectors of equal dimension u .

When modular vector arithmetic is considered, assume

$$\vec{x}, \vec{y} \in \mathbb{Z}_v^u,$$

where $v > 0$.

Let the signed-pair representation of the common modulus v used for coordinate i be

$$(P_{a_i}, P_{b_i}),$$

with

$$\boxed{\Lambda[(P_{a_i}, P_{b_i})] \equiv v.}$$

1. Vector Addition Before Modular Reduction

Since each coordinate x_i and y_i is a numerical symbol, both must first be converted into signed-pair form before being supplied to Γ .

For each coordinate i , define the pre-modular sum

$$\boxed{\bar{z}_i = \Gamma \left(\Lambda^{-1}(x_i), \Lambda^{-1}(y_i) \right) .}$$

Therefore,

$$\overrightarrow{\vec{x} + \vec{y}} = \begin{pmatrix} \Gamma(\Lambda^{-1}(x_1), \Lambda^{-1}(y_1)) \\ \Gamma(\Lambda^{-1}(x_2), \Lambda^{-1}(y_2)) \\ \vdots \\ \Gamma(\Lambda^{-1}(x_u), \Lambda^{-1}(y_u)) \end{pmatrix}.$$

Each coordinate of

$$\overrightarrow{\vec{x} + \vec{y}}$$

is a reduced signed pair produced by Γ .

No modular reduction has yet been applied.

2. Vector Addition Modulo v

To return each coordinate to $\mathbb{Z}_v$, apply Ξ using the corresponding modulus pair and extract the remainder.

Thus,

$$\vec{x} + \vec{y} = \begin{pmatrix} \text{rem}(\Xi[\Gamma(\Lambda^{-1}(x_1), \Lambda^{-1}(y_1)), (P_{a_1}, P_{b_1})]) \\ \text{rem}(\Xi[\Gamma(\Lambda^{-1}(x_2), \Lambda^{-1}(y_2)), (P_{a_2}, P_{b_2})]) \\ \vdots \\ \text{rem}(\Xi[\Gamma(\Lambda^{-1}(x_u), \Lambda^{-1}(y_u)), (P_{a_u}, P_{b_u})]) \end{pmatrix}.$$

Therefore,

$$\vec{x} + \vec{y} \in \mathbb{Z}_v^u.$$

The distinction is

$$\overrightarrow{\vec{x} + \vec{y}} = \text{coordinate-wise signed addition before modulo}$$

while

$$\vec{x} + \vec{y} = \text{coordinate-wise signed addition after modulo } v.$$

3. Scalar-Vector Multiplication Before Modular Reduction

Let

δ

be a signed numerical symbol.

Since Υ requires signed-pair inputs, convert both the scalar and each vector coordinate using Λ^{-1} .

For each coordinate x_i , define

$$\boxed{\overline{w}_i = \Upsilon [\Lambda^{-1}(\delta), \Lambda^{-1}(x_i)] .}$$

Because Υ returns a signed numerical symbol, each coordinate of the pre-modular scalar-vector product is a signed symbol.

Therefore,

$$\boxed{\overrightarrow{\delta x} = \begin{pmatrix} \Upsilon [\Lambda^{-1}(\delta), \Lambda^{-1}(x_1)] \\ \Upsilon [\Lambda^{-1}(\delta), \Lambda^{-1}(x_2)] \\ \vdots \\ \Upsilon [\Lambda^{-1}(\delta), \Lambda^{-1}(x_u)] \end{pmatrix} .}$$

At this stage no modular reduction has been applied.

Thus,

$$\boxed{\overrightarrow{\delta x} = \text{scalar-vector multiplication before modulo } v.}$$

4. Scalar-Vector Multiplication Modulo v

To apply Ξ in its signed-pair form, each signed-symbol output of Υ must first be converted back into a signed pair using Λ^{-1} .

For each coordinate,

$$\Lambda^{-1} (\Upsilon [\Lambda^{-1}(\delta), \Lambda^{-1}(x_i)])$$

is the canonical signed-pair representation of the pre-modular product.

Therefore define

$$\delta \vec{x} = \begin{pmatrix} \text{rem}(\Xi[\Lambda^{-1}(\Upsilon[\Lambda^{-1}(\delta), \Lambda^{-1}(x_1)]), (P_{a_1}, P_{b_1})]) \\ \text{rem}(\Xi[\Lambda^{-1}(\Upsilon[\Lambda^{-1}(\delta), \Lambda^{-1}(x_2)]), (P_{a_2}, P_{b_2})]) \\ \vdots \\ \text{rem}(\Xi[\Lambda^{-1}(\Upsilon[\Lambda^{-1}(\delta), \Lambda^{-1}(x_u)]), (P_{a_u}, P_{b_u})]) \end{pmatrix}.$$

Hence,

$$\delta \vec{x} \in \mathbb{Z}_v^u.$$

The distinction is therefore

$$\overline{\delta \vec{x}} = \text{scalar multiplication before coordinate-wise modular reduction}$$

and

$$\delta \vec{x} = \text{scalar multiplication after coordinate-wise modular reduction.}$$

5. Type Summary

For vector addition,

$$x_i \xrightarrow{\Lambda^{-1}} \text{signed pair}, \quad y_i \xrightarrow{\Lambda^{-1}} \text{signed pair},$$

followed by

$$\Gamma : (\text{signed pair}, \text{signed pair}) \longrightarrow \text{signed pair}.$$

Thus each coordinate of

$$\overline{\vec{x} + \vec{y}}$$

is a signed pair.

For scalar-vector multiplication,

$$\delta \xrightarrow{\Lambda^{-1}} \text{signed pair}, \quad x_i \xrightarrow{\Lambda^{-1}} \text{signed pair},$$

followed by

$$\Upsilon : (\text{signed pair}, \text{signed pair}) \longrightarrow \text{signed symbol}.$$

Thus each coordinate of

$$\overline{\delta \vec{x}}$$

is a signed numerical symbol.

Before pair-form Ξ can be applied to such a coordinate, the coordinate is converted by

$$\Lambda^{-1} : \text{signed symbol} \longrightarrow \text{canonical signed pair}.$$

Finally,

$$\text{rem}(\Xi(\cdot))$$

returns the numerical coordinate reduced modulo v .

Dot Product of Two Vectors

Let

$$\vec{x} = (x_1, x_2, \dots, x_u)$$

and

$$\vec{y} = (y_1, y_2, \dots, y_u)$$

be vectors of equal dimension u .

For each coordinate i , first convert the numerical symbols

$$x_i \quad \text{and} \quad y_i$$

into canonical signed-pair representations using Λ^{-1} .

Then multiply the corresponding coordinates using Υ .

Since Υ returns a signed numerical symbol, convert that output back into a signed pair using Λ^{-1} .

Define

$$T_i = \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_i), \Lambda^{-1}(y_i) \right] \right).$$

Thus each T_i is the signed-pair representation of the product

$$x_i y_i.$$

For three coordinates,

$$\vec{x} = (x_1, x_2, x_3)$$

and

$$\vec{y} = (y_1, y_2, y_3),$$

the three coordinate products are

$$T_1 = \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_1), \Lambda^{-1}(y_1) \right] \right),$$

$$T_2 = \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_2), \Lambda^{-1}(y_2) \right] \right),$$

and

$$T_3 = \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_3), \Lambda^{-1}(y_3) \right] \right).$$

Since Γ accepts two signed-pair inputs and returns one signed-pair output, the products may be added recursively.

First,

$$\Gamma(T_1, T_2)$$

produces the signed-pair representation of

$$x_1 y_1 + x_2 y_2.$$

That output may then be supplied directly as an input to another application of Γ .

Therefore,

$$\boxed{\vec{x} \cdot \vec{y} = \Gamma \left(\Gamma(T_1, T_2), T_3 \right) .}$$

Substituting the definitions of T_1, T_2, T_3 ,

$$\overrightarrow{\vec{x} \cdot \vec{y}} = \Gamma \left(\Gamma \left(\Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_1), \Lambda^{-1}(y_1) \right] \right), \right. \right. \\ \left. \left. \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_2), \Lambda^{-1}(y_2) \right] \right) \right), \right. \\ \left. \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_3), \Lambda^{-1}(y_3) \right] \right) \right).$$

The overbar indicates that the dot product has been computed, but modular reduction has not yet been applied.

More generally, for vectors of dimension u ,

$$\vec{x} = (x_1, x_2, \dots, x_u)$$

and

$$\vec{y} = (y_1, y_2, \dots, y_u),$$

define

$$T_i = \Lambda^{-1} \left(\Upsilon \left[\Lambda^{-1}(x_i), \Lambda^{-1}(y_i) \right] \right)$$

for

$$i = 1, 2, \dots, u.$$

Then define the pre-modular dot product by repeated application of Γ :

$$\overrightarrow{\vec{x} \cdot \vec{y}} = \Gamma \left(\dots \Gamma \left(\Gamma(T_1, T_2), T_3 \right) \dots, T_u \right).$$

Equivalently, define the partial sums recursively by

$$D_1 = T_1,$$

and for

$$i = 2, 3, \dots, u,$$

define

$$\boxed{D_i = \Gamma(D_{i-1}, T_i).$$

Then

$$\boxed{\overrightarrow{x} \cdot \overrightarrow{y} = D_u.}$$

Thus

$$\overrightarrow{x \cdot y}$$

is a reduced signed-pair representation of

$$x_1y_1 + x_2y_2 + \cdots + x_uy_u$$

before modular reduction.

If

$$\vec{x}, \vec{y} \in \mathbb{Z}_v^u,$$

and

$$(P_a, P_b)$$

is a signed-pair representation of the positive modulus v , so that

$$\Lambda[(P_a, P_b)] \equiv v,$$

then the modular dot product is defined by

$$\boxed{\vec{x} \cdot \vec{y} = \text{rem} \left(\Xi \left[\overrightarrow{x \cdot y}, (P_a, P_b) \right] \right) .}$$

Therefore,

$$\boxed{\overrightarrow{x \cdot y} = \text{dot product before modular reduction}}$$

while

$$\boxed{\vec{x} \cdot \vec{y} = \text{dot product after reduction modulo } v.}$$

Matrix Addition

Let

$$A = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{pmatrix}$$

and

$$B = \begin{pmatrix} B_{11} & B_{12} & \cdots & B_{1n} \\ B_{21} & B_{22} & \cdots & B_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mn} \end{pmatrix}$$

be two matrices of equal dimensions $m \times n$.

Each entry is a signed-pair representation.

Thus, for every row i and column j ,

$$A_{ij} = (P_{a_{ij}}, P_{b_{ij}})$$

and

$$B_{ij} = (P_{c_{ij}}, P_{d_{ij}}).$$

Matrix addition is performed coordinate-wise.

Since Γ accepts two signed-pair inputs and returns one reduced signed-pair output, the pre-modular sum of the entries in position (i, j) is

$$\boxed{\overline{C}_{ij} = \Gamma(A_{ij}, B_{ij}).}$$

Equivalently,

$$\boxed{\overline{C}_{ij} = \Gamma((P_{a_{ij}}, P_{b_{ij}}), (P_{c_{ij}}, P_{d_{ij}})).}$$

Therefore the pre-modular matrix sum is

$$\overline{A+B} = \begin{pmatrix} \Gamma(A_{11}, B_{11}) & \Gamma(A_{12}, B_{12}) & \cdots & \Gamma(A_{1n}, B_{1n}) \\ \Gamma(A_{21}, B_{21}) & \Gamma(A_{22}, B_{22}) & \cdots & \Gamma(A_{2n}, B_{2n}) \\ \vdots & \vdots & \ddots & \vdots \\ \Gamma(A_{m1}, B_{m1}) & \Gamma(A_{m2}, B_{m2}) & \cdots & \Gamma(A_{mn}, B_{mn}) \end{pmatrix}.$$

Every entry of

$$\overline{A+B}$$

is therefore a reduced signed-pair output produced by Γ .

At this stage, no modular reduction has been applied.

Thus,

$$\boxed{\overline{A+B} = \text{matrix addition before modular reduction.}}$$

If

$$A, B \in \mathbb{Z}_v^{m \times n},$$

then each matrix entry must be reduced modulo the same positive modulus v .

Let

$$(P_{p_{ij}}, P_{q_{ij}})$$

be a signed-pair representation of v for entry (i, j) , so that

$$\boxed{\Lambda[(P_{p_{ij}}, P_{q_{ij}})] \equiv v.}$$

Since

$$\Gamma(A_{ij}, B_{ij})$$

already returns a signed pair, it may be supplied directly to pair-form Ξ .

Therefore define the modular matrix-sum entry by

$$\boxed{C_{ij} = \text{rem} \left(\Xi \left[\Gamma(A_{ij}, B_{ij}), (P_{p_{ij}}, P_{q_{ij}}) \right] \right) .}$$

Thus the modular matrix sum is

$$A + B = \begin{pmatrix} \text{rem}(\Xi[\Gamma(A_{11}, B_{11}), (P_{p_{11}}, P_{q_{11}})]) & \cdots & \text{rem}(\Xi[\Gamma(A_{1n}, B_{1n}), (P_{p_{1n}}, P_{q_{1n}})]) \\ \vdots & \ddots & \vdots \\ \text{rem}(\Xi[\Gamma(A_{m1}, B_{m1}), (P_{p_{m1}}, P_{q_{m1}})]) & \cdots & \text{rem}(\Xi[\Gamma(A_{mn}, B_{mn}), (P_{p_{mn}}, P_{q_{mn}})]) \end{pmatrix}.$$

Each entry of

$$A + B$$

is now a numerical symbol in

$$\{0, 1, \dots, v - 1\}.$$

Therefore,

$$A + B \in \mathbb{Z}_v^{m \times n}.$$

The distinction is

$$\overline{A + B} = \text{coordinate-wise signed-pair matrix addition before modulo } v$$

while

$$A + B = \text{coordinate-wise matrix addition after modulo } v.$$

The type flow for each matrix entry is

$$(\text{signed pair}, \text{signed pair}) \xrightarrow{\Gamma} \text{reduced signed pair}$$

followed, when modular reduction is required, by

$$\text{reduced signed pair} \xrightarrow{\Xi} (q, r) \xrightarrow{\text{rem}} r.$$